AI702 Lab 0 Pre-lab

Computing environment, arrays and experimental evidence

Review array indexing, matrix multiplication, broadcasting, Python classes, tensors, gradients and reproducibility. **Complete Problems 1 and 2** and submit your answers on Moodle LMS as part of the overall zip file that you will submit after doing actual lab and post-lab. **Show intermediate calculations and state any assumptions.**

Problem 1 Shapes and silent errors

(a) A data matrix has shape (6, 4). The first dimension corresponds to the number of samples and the second sample corresponds to the number of features per sample. A linear transformation uses two selected features and produces three scores per example. State the shapes of the selected matrix, the weight matrix in (output, input) order, the bias vector and the result. Repeat the shape trace for a batch containing only one example. Explain why preserving the leading singleton dimension (the batch dimension) is important when the batch size is one.

(b) Validation examples must be standardized using statistics estimated from the training split. Explain what changes if statistics are instead recomputed from the validation examples. Specify a check that would detect accidental mutation of the original data during standardization.

(c) Consider y = [[0], [1], [1], [0]] and p = [0, 1, 0, 0], represented as NumPy arrays. Determine the shape and value of (y == p).mean(). Compare the corresponding entries in the two arrays and compute the rate of matching entries. Explain why checking only that a reported accuracy lies between zero and one would miss this error.

Problem 2 Reproducibility

(a) Two experiment reports show the same rounded accuracy. What would you need to check to determine whether they used the same examples, predictions, settings and code? Explain why an accuracy value or a code version alone is not enough.

(b) A reported accuracy is numerically plausible but has no stated unit or record of how rows were matched. Propose two independent checks that could establish whether it is valid. Explain which type of error each check would detect.

Preparation of your computer

Use uv and Python 3.12 for the exercises. On macOS or Linux, install uv with:

```
curl -LsSf https://astral.sh/uv/0.12.18/install.sh | sh
```

Open a new terminal, then run:

```
uv --version
uv python install 3.12
```

Next, create a virtual environment using Python 3.12:

```
uv venv --python 3.12
```

Install NumPy using uv:

```
uv pip install numpy
```

Install the CPU-only version of PyTorch:

```
uv pip install torch --torch-backend cpu
```

Verify that NumPy and PyTorch are installed correctly:

```
uv run --no-project python -c "import numpy; print('NumPy:', numpy.__version__)"
uv run --no-project python -c "import torch; print('PyTorch:', torch.__version__);
print('CPU test:', torch.rand(2, 3))"
```

Windows users should follow https://docs.astral.sh/uv/getting-started/installation/ and the TA's environment instructions.

These examples use a macOS or Linux shell. All Python commands use uv, so no separate system command named `python` is needed.